Java IO

What will be covered

OBJECT™ TECHNOLOGIES

- What is stream
- IO Stream and its types
- Byte Streams
- Character Streams
- Standard - built-in streams
- Reading from console
- File streams
- File class
- Reading data from file
- Writing data to file
- Buffered stream
- Flushing buffered straems
- Serialization
- Object streams
- Use of serialization
- Example
- Desrialization
- Externalizable

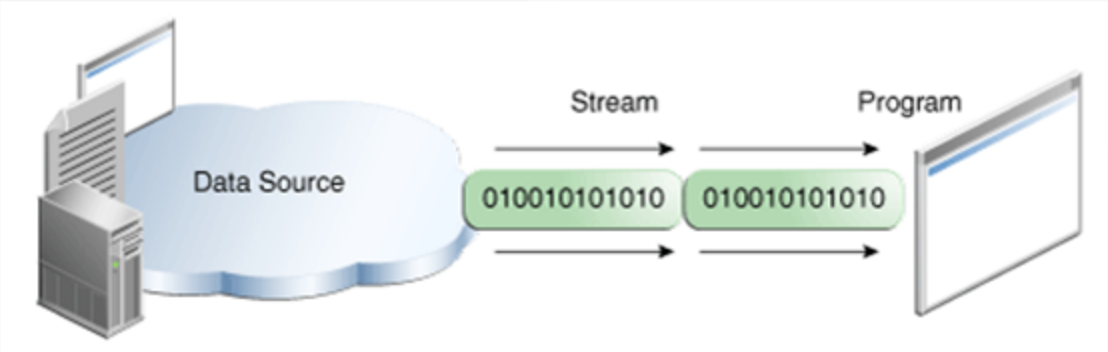
What is stream

- An *I/O Stream* represents an input source or an output destination.
- A stream can represent many different kinds of sources and destinations, including disk files, devices, other programs, and memory arrays.
- Streams support many different kinds of data, including simple bytes, primitive data types, localized characters, and objects. Some streams simply pass on data; others manipulate and transform the data in useful ways.
- All streams present the same simple model to programs that use them: A stream is a sequence of data.
- A program uses an *input stream* to read data from a source, one item at a time. A program uses an *output stream* to write data to a destination, one item at time.

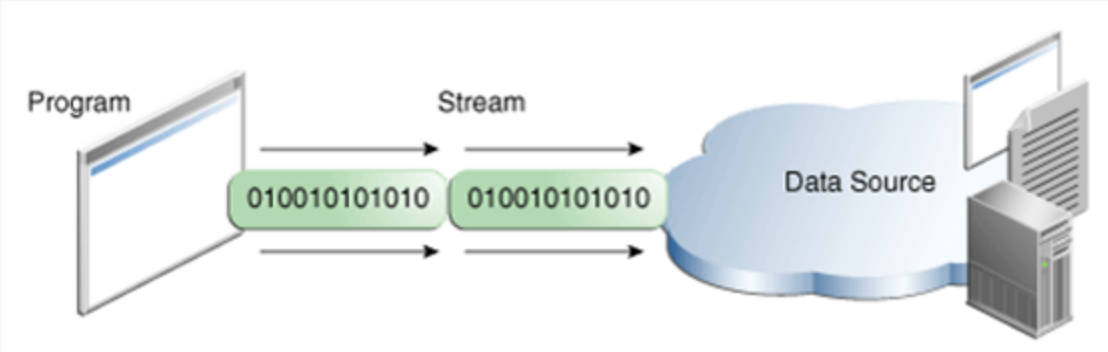
www.object.co.in

Reading and writing streams

- **Reading the data from the source**



- **Writing the data to the destination**



www.object.co.in

IO Stream

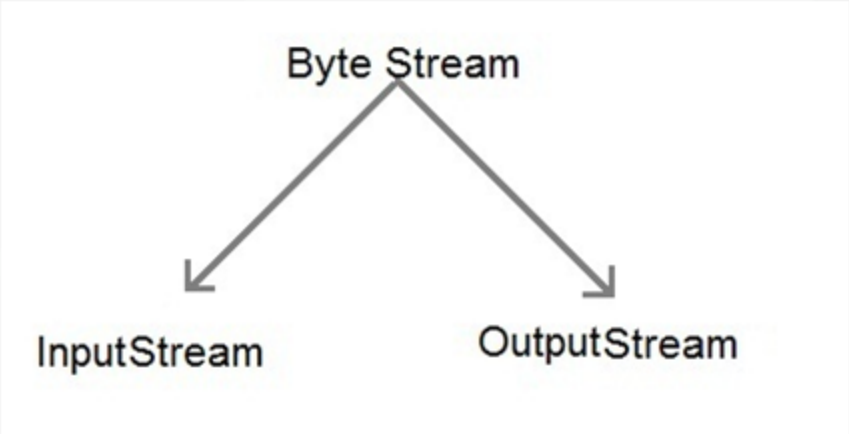
OBJECT™ TECHNOLOGIES

- Java performs I/O through **Streams**. A Stream is linked to a physical layer by java I/O system to make input and output operation in java.
- Streams are clean way to deal with input/output without having every part of your code understand the physical.
- Java encapsulates Stream under **java.io** package. Java defines two types of streams. They are,
  - **Byte Stream :** It provides a convenient means for handling input and output of byte.
  - **Character Stream :** It provides a convenient means for handling input and output of characters. Character stream uses Unicode and therefore can be internationalize

www.object.co.in

Byte Stream Classes

- Byte stream is defined by using two abstract class at the top of hierarchy, they are InputStream and OutputStream
- These two abstract classes have several concrete classes that handle various devices such as disk files, network connection etc.



- Some important Byte stream classes
  - BufferedInputStream : BufferedOutputStream.
  - DataInputStream : DataOutputStream
  - FileInputStream : FileOutputStream
  - ObjectInputStream : ObjectOutputStream
  - PrintStream

www.object.co.in

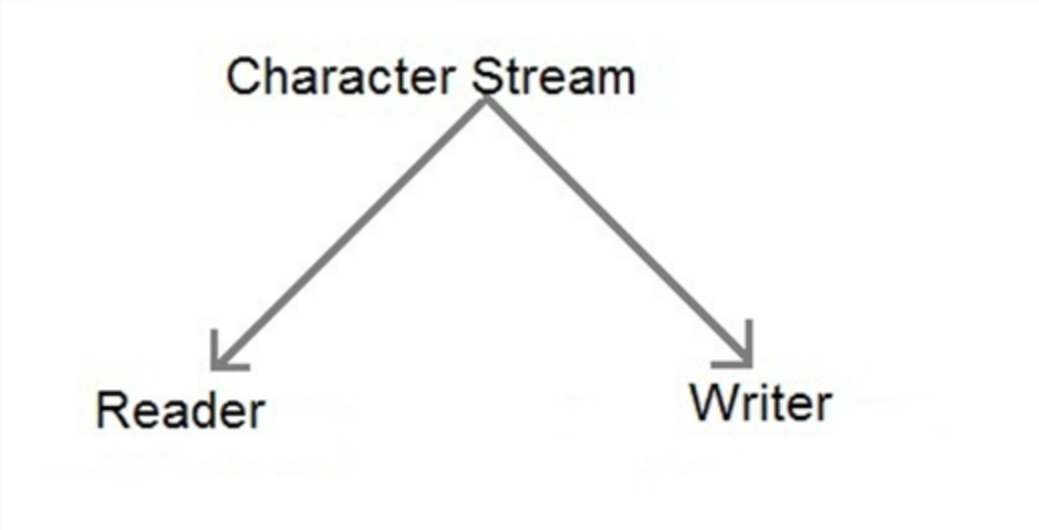
Important methods

Methods of InputStream	Methods of OutputStream
int available()	void close()
void close()	void flush()
void mark(int readlimit)	void write(byte[] b)
boolean markSupported()	void write(byte[] b, int off, int len)
abstract int read()	abstract void write(int b)
int read(byte[] b)	
int read(byte[] b, int off, int len)	
void reset()	
long skip(long n)	

www.object.co.in

Character Stream Classes

- The character streams were designed after the byte streams to provide full support for working with Unicode characters
- Character stream is also defined by using two abstract class at the top of hierarchy, they are Reader and Writer.
- These two abstract classes have several concrete classes that handle unicode character.



www.object.co.in

Character Stream Classes

- Some important Character stream classes
  - BufferedReader : BufferedWriter
  - FileReader : FileWriter
  - InputStreamReader : OutputStreamReader
  - PrintWriter

www.object.co.in

Important methods

Methods of Reader	Methods of Writer
int available()	abstract void close()
void close()	abstract void flush()
void mark(int readlimit)	void write(char [] b)
boolean markSupported()	abstract void write(char [] b, int off, int len)
int read()	void write(int b)
int read(char [] b)	void write(String str)
abstract int read(char[] b, int off, int len)	void write(String str, int off, int len)
void reset()	
long skip(long n)	

www.object.co.in

Standard streams

OBJECT™ TECHNOLOGIES

- A program is often run from the command line and interacts with the user in the command line environment. The Java platform supports this kind of interaction in two ways: through the Standard Streams and through the Console.
- The Java platform supports three Standard Streams: *Standard Input*, accessed through System.in; *Standard Output*, accessed through System.out; and *Standard Error*, accessed through System.err.
- These objects are defined automatically and do not need to be opened. These are static objects of class System.
- System.out and System.err are defined as PrintStream objects.

Reading Console Input

Object of BufferedReader class

```
BufferedReader br = new BufferedReader(new
                                InputStreamReader (System.in) );
```

InputStreamReader is subclass of Reader class. It converts bytes to character.

Console inputs are read from this.

- System.in is an **InputStream** which is typically connected to keyboard input of console programs
- The **Java.io.InputStreamReader** class is a bridge from byte streams to character streams.It reads bytes and decodes them into characters using a specified charset

www.object.co.in

Reading from console

```
char c;
BufferedReader br = new
BufferedReader(new InputStreamReader(System.in));
System.out.println("Enter characters, 'q' to quit.");
// read characters
do {
c = (char) br.read();
System.out.println(c);
} while(c != 'q');
}
```

Reads a single character at a time

```
BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));
System.out.print("Please enter your name? ");
String name = reader.readLine();
System.out.println("Your name is: " + name);
```

Reading strings

www.object.co.in

File Streams

OBJECT™ TECHNOLOGIES

- Following classes can be used for reading a file
  - FileInputStream
  - FileReader
- Following stream classes can be used for writing to file
  - FileOutputStream
  - FileWriter
- All the classes can take file path as argument or File instance as argument.
- File class is used for getting meta information about the file
- All the reading streams may throw FileNotFoundException if file is not found but all the writing streams will create a file if file is not found.

  Note: only file can be created in the specified path but if path is not available it will throw the exception

File class

- The File object represents the actual file/directory on the disk.
- Java File class represents the files and directory pathnames in an abstract manner.
- This class is used for creation of files and directories, file searching, file deletion, etc.
- There are many helper methods which can be used to manipulate the files.
  - **public boolean isFile()**
  - **public boolean exists()**
  - **public String getAbsolutePath()**
  - **public long lastModified()**
  - **public long length()**
  - **public String[] list()**

www.object.co.in

Get information about file

```
//accept file name or directory name through command line args
String fname =args[0];

//pass the filename or directory name to File object
File f = new File(fname);

//apply File class methods on File object
System.out.println("File name :"+f.getName());
System.out.println("Path: "+f.getPath());
System.out.println("Absolute path:" +f.getAbsolutePath());
System.out.println("Parent:"+f.getParent());
System.out.println("Exists :"+f.exists());
if(f.exists())
{
    System.out.println("Is writeable:"+f.canWrite());
    System.out.println("Is readable"+f.canRead());
    System.out.println("Is a directory:"+f.isDirectory());
    System.out.println("File Size in bytes "+f.length());
}
```


www.object.co.in

Reading from file

- Java FileInputStream class obtains input bytes from a file. It is used for reading byte-oriented data (streams of raw bytes) such as image data, audio, video etc.
- You can also read character-stream data. But, for reading streams of characters, it is recommended to use FileReader class.

```java
File file = new File("C:/robots.txt");
FileInputStream fis = null;

try {
    fis = new FileInputStream(file);

    System.out.println("Total file size to read (in bytes) : "
            + fis.available());

    int content;
    while ((content = fis.read()) != -1) {
        // convert to char and display it
        System.out.print((char) content);
    }

} catch (IOException e) {
    e.printStackTrace();
} finally {
    try {
        if (fis != null)
            fis.close();
    } catch (IOException ex) {
        ex.printStackTrace();
    }
}
```

Reading from file

- Note : File path can be specified using either forward slash(/) or using double backward slash(\\)
- Generally resources are closed in finally block. But from java 7 we can use :

```
File file = new File("C:/robots.txt");

try (FileInputStream fis = new FileInputStream(file)) {

    System.out.println("Total file size to read (in bytes) : "+ fis.available());

    int content;
    while ((content = fis.read()) != -1) {
        // convert to char and display it
        System.out.print((char) content);
    }

} catch (IOException e) {
    e.printStackTrace();
}
```


www.object.co.in

Writing to file

- Java FileOutputStream is an output stream used for writing data to a file.
- If you have to write primitive values into a file, use FileOutputStream class. You can write byte-oriented as well as character-oriented data through FileOutputStream class.
- But, for character-oriented data, it is preferred to use FileWriter than FileOutputStream.
- In the constructors of both the classes some constructors are accepting boolean value which indicates whether file contents are should be appended or not

Writing to file

```
public static void main(String args[])throws IOException {
   File file = new File("Hello1.txt");

   // creates the file
   file.createNewFile();

   // creates a FileWriter Object
   FileWriter writer = new FileWriter(file);

   // Writes the content to the file
   writer.write("This\n is\n an\n example\n");
   writer.flush();
   writer.close();

   // Creates a FileReader Object
   FileReader fr = new FileReader(file);
   char [] a = new char[50];
   fr.read(a);   // reads the content to the array

   for(char c : a)
      System.out.print(c);   // prints the characters one by one
   fr.close();
}
```


www.object.co.in

Buffered streams

OBJECT™ TECHNOLOGIES

- Each read or write request is handled directly by the underlying OS.
- This can make a program much less efficient, This can make a program much less efficient, since each such request often triggers disk access, network activity, or some other operation that is relatively expensive.
- To reduce this kind of overhead, the Java platform implements *buffered* I/O streams.
- Buffered input streams read data from a memory area known as a *buffer*; the native input API is called only when the buffer is empty.
- Similarly, buffered output streams write data to a buffer, and the native output API is called only when the buffer is full
- Buffered stream classes :
  - BufferedInputStream
  - BufferedOutputstream
  - BufferedReader
  - BufferedWriter

www.object.co.in

Buffered Stream - example

```
//create file object
File file = new File("C://FileIO//ReadFile.txt");
BufferedInputStream bin = null;

try
{
    //create FileInputStream object
    FileInputStream fin = new FileInputStream(file);

    //create object of BufferedInputStream
    bin = new BufferedInputStream(fin);

    //create a byte array
    byte[] contents = new byte[1024];

    int bytesRead=0;
    String strFileContents;

    while( (bytesRead = bin.read(contents)) != -1){

        strFileContents = new String(contents, 0, bytesRead);
        System.out.print(strFileContents);
    }
```

Actual read operation is performed by buffered stream. For creating buffered stream object instance of File stream is passed as an argument

www.object.co.in

Buffered Stream - example

```
public static void main(String[] args) throws Exception {
    FileWriter writer = new FileWriter("D:\\testout.txt");
    BufferedWriter buffer = new BufferedWriter(writer);
    buffer.write("Welcome to javaTpoint.");
    buffer.close();
    System.out.println("Success");
    }
}
```

- Writing operation is performed by buffered stream
- It will write the data into the buffer and when buffer is full then only it is written in the underlying stream i.e. file.
- Size of the buffer can be configured at the time of creating buffered streams for performance optimization.

www.object.co.in

Flushing Buffered Streams

OBJECT™ TECHNOLOGIES

- It often makes sense to write out a buffer at critical points, without waiting for it to fill. This is known as *flushing* the buffer.
- Some buffered output classes support *autoflush*, specified by an optional constructor argument. When autoflush is enabled, certain key events cause the buffer to be flushed. For example, an autoflush PrintWriter object flushes the buffer on every invocation of println or format.
- To flush a stream manually, invoke its flush method.
- The flush method is valid on any output stream, but has no effect unless the stream is buffered.

www.object.co.in

Serialization

OBJECT™ TECHNOLOGIES

- To *serialize* an object means to convert its state to a byte stream so that the byte stream can be reverted back into a copy of the object.
- A Java object is *serializable* if its class or any of its superclasses implements either the java.io.Serializable interface or its subinterface, java.io.Externalizable.
- *Deserialization* is the process of converting the serialized form of an object back into a copy of the object.
- All the instance variables are saved during serialization
- Generally derived values are not serialized, if any instance member is to be excluded from serialization, it can be marked as transient.
- If Super class implements Serializable then sub class are also Serializable automatically.

Object streams

OBJECT™ TECHNOLOGIES

- object streams support I/O of objects.
- Most, but not all, standard classes support serialization of their objects.
- Those that do implement the marker interface Serializable.
- The object stream classes are ObjectInputStream and ObjectOutputStream. These classes implement ObjectInput and ObjectOutput, which are subinterfaces of DataInput and DataOutput.
- That means that all the primitive data I/O methods are also implemented in object streams. So an object stream can contain a mixture of primitive and object values.
- The **Java.io.ObjectOutputStream** class writes primitive data types and graphs of Java objects to an OutputStream.The objects can be read (reconstituted) using an ObjectInputStream

Use of Serialization

- serialization will translate the Object state to Byte Streams. This Byte stream can be used for different purpose.
  - Write to Disk
  - Store in Memory
  - Sent byte stream to other platform over network
  - Save byte stream in DB(As BLOB)
- It is mainly used in Hibernate, RMI, JPA, EJB and JMS technologies.
- It is mainly used to travel object's state on the network (known as marshaling).

www.object.co.in

Serialization example

```
import java.io.*;
class Persist{
 public static void main(String args[])throws Exception{
  Student s1 =new Student(211,"ravi");

  FileOutputStream fout=new FileOutputStream("f.txt");
  ObjectOutputStream out=new ObjectOutputStream(fout);

  out.writeObject(s1);
  out.flush();
  System.out.println("success");
 }
}
```

- Note : If Student class does not implement Serializable interface, it will throw NotSerializableException at run time

www.object.co.in

Deserialization example

```
import java.io.*;
class Depersist{
 public static void main(String args[])throws Exception{

  ObjectInputStream in=new ObjectInputStream(new FileInputStream("f.txt"));
  Student s=(Student)in.readObject();
  System.out.println(s.id+" "+s.name);

  in.close();
 }
}
```

- Note : readObject() method returns Object that has to be upcasted to an appropriate type. If needed it can be supported with

www.object.co.in

Externalizable in java

- The Externalizable interface provides the facility of writing the state of an object into a byte stream in compress format. It is not a marker interface.
- The Externalizable interface provides two methods:
  - **public void writeExternal(ObjectOutput out) throws IOException**
  - **public void readExternal(ObjectInput in) throws IOException**

www.object.co.in